

PROF. DR.-ING. MARK SCHUTERA

_SUPERVISED LEARNING

UNFINISHED LECTURE NOTES

Copyright © 2026 Prof. Dr.-Ing. Mark Schutera

PUBLISHED BY UNFINISHED LECTURE NOTES

Combobulated with the help of multiple large language model driven tools. Licensed under the Creative Commons Attribution-NonCommercial 4.0 International License ("CC BY-NC-SA 4.0"). You may not use this file for commercial purposes. If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original You must obtain explicit permission from the author for uses beyond those permitted by this license. To view a copy of this license, visit <https://creativecommons.org/licenses/by-nc-sa/4.0/>. Unless required by applicable law or agreed to in writing, distributed material is provided on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the license for details.

These notes are, by their very nature, unfinished, and they improve with every reader. If you spot an error, disagree with a framing, or want to add a source, a question, or a position, open a pull request at https://github.com/Quillstacks/LectureMaterial/tree/main/lecturenotes/notes_unsupervisedlearning. Every contribution is welcome.



Last updated, May 2026 — v0.0.10

"IN MY DEFENSE, I WAS LEFT UNSUPERVISED".

DER ZAUBERLEHRLING

Contents

<i>I</i>	<i>Geometry of Pattern</i>	11
<i>II</i>	<i>Representing Pattern</i>	13
<i>III</i>	<i>Distilling Pattern</i>	15
	<i>Transfer Learning</i>	17
	<i>Self-Supervised Learning</i>	21
	<i>Weak Supervision</i>	27
<i>IV</i>	<i>Interaction Pattern</i>	31
	<i>Reinforcement Learning Fundamentals</i>	33
	<i>Index</i>	39

Introduction

SUPERVISED LEARNING is limited to tasks where we have labeled data and clear objectives. Beyond this limited context of learning, we fall back to the pattern in the data itself.

WE EXPLORE HOW PATTERN EMERGES, from other sources of information beyond explicit labels. We will see how pattern can be discovered in data, represented, leveraged and utilized. In that sense this course is about any form of learning that is not supervised learning.

EACH LECTURE FOLLOWS A CONSISTENT PATTERN. To start of the contents of the lecture will be motivated by a narrative, a theoretical primer, and some sort of hands-on experience to build intuition. Then we will dive into the, theory and algorithms, then reflection. Theory sessions are 45 minutes, followed by 45 minutes of coding practice to enforce reflection and deepen understanding of the concepts.

A Philosophy of Education

THE BEAUTIFUL RISK OF EDUCATION is that it is an encounter. I am trying to model a way of thinking, and reasoning, then stepping back. The rest is yours.

- **Start with why.** A method understood only procedurally is a method you will misapply when it matters. Before learning how something works, understand why it was built, what it replaced, and where it fails. The mechanics then follow from necessity, not memorization.
- **Build taste alongside competence.** You need a sense of what beautiful is, you need to train your aesthetics before you can reliably produce it. This develops by studying what others have built, making it your own, then making your own, and honestly evaluating the gap between them. Taste is the compass. Skill is the engine. You need both, in that order.
- **Failure is the designed medium.** Each attempt teaches what a clean run cannot. The insight compounds across tries, not from a single correct execution. Those who never start failing will fail.
- **Work in the open.** Showing half-finished thinking, naming mistakes as they happen, is a professional skill. We practice it here. It is how knowledge actually moves between people. And it is how knowledge moves towards you, at unprecedented speed.
- **The fallacy of mediocracy.** Iteration and failure are the method, not the excuse for settling. Do not confuse velocity with quality. Whatever you learn, whatever you build: make it good.
- **Outcomes emerge. They are not delivered.** Judgment, taste, the capacity to sit with uncertainty: these are not course outputs. They are what you build from the encounter. I can create the conditions. The rest depends on what you do with them.

Part I

Geometry of Pattern

Part II

Representing Pattern

Part III

Distilling Pattern

Transfer Learning

THE FLYWHEEL OF GENERALIZED EMBEDDINGS.

The Why

IN THE CHAPTER ON VARIATIONAL AUTO ENCODERS WE TRAINED AN ENCODER that mapped inputs into a structured latent space, where nearby points were similar and the bottleneck, our embedding, distilled what mattered. The space was useful, but it belonged to one model trained for one task. By the same logic, every new task would mean training that encoder again, from scratch, on a freshly and sufficiently labelled dataset.

LET'S REUSE OR TRANSFER EMBEDDINGS. Almost no production model is trained from scratch. They start from a backbone someone else pretrained at scale, freeze or adapt it, and bolt a small task-specific head on top. The encoder is the asset, the head is a thin wrapper, for a narrow task or domain, and the same backbone is shared across many downstream tasks.

TRAINING AN ENCODER PER TASK IS NOT VIABLE. Labels are expensive to collect, domains splinter into modalities that each deserve their own representation, pretraining a competitive backbone costs hundreds of GPU years, and entire classes of inputs have no labelled examples to begin with. The encoder therefore has to come from somewhere else and the cost of obtaining it has to be amortised across many downstream tasks.

REUSE IS THE MECHANISM. A backbone trained at scale on a broad task supplies features that work, often surprisingly well, for tasks it was never trained on. Push the scale far enough and the backbone becomes a foundation model that handles new tasks without any task-specific training at all, because its embeddings already cover the world that the new task lives in. Once we found that the long tail of

MACHINE LEARNING ENGINEER

DEEP LEARNING ENGINEER

TRAIN ONCE, REUSE FOREVER. Almost every modern vision and language system in production starts from someone else's pretrained encoder. The encoder is the expensive part, the task-specific head on top is cheap, and the choice of encoder determines almost everything that follows.

SCARCE LABELS. A medical imaging team may need months to label five hundred chest X-rays, while a competitive vision encoder is trained on millions.

SPLINTERED DOMAINS. Satellite imagery, manufacturing defects, document layouts, and X-ray modalities each have their own vocabulary of inputs and in principle each deserves its own encoder.

ENORMOUS COMPUTE. Pretraining a modern vision transformer or large language model costs hundreds of GPU years, which few teams can afford once and none can afford per task.

any one task already sits inside the backbone's embedding ¹.

ON MODALITIES. Foundation models such as CLIP, GPT, and SAM push this idea to the extreme: A single representation, learned once on hundreds of millions of image-text pairs or tokens, can be reused on classification, retrieval, or segmentation tasks it was never explicitly trained for, with zero or only a handful of labelled examples. Natural language is specifically applicable to task transfer as the input itself is adaptable and directs the task. The same property that makes the input the task specification is also the surface on which prompt injection attacks operate, since adversarial text inserted into the input can redirect the model to a task its operator never intended ².

IN ORDER TO MOVE FROM ONE-TASK-ONE-MODEL to many-tasks-one-backbone we have good reason to find ways to,

- reuse features learned at scale for downstream tasks with little or no labelled data.
- adapt pretrained representations efficiently, without paying full retraining cost.
- bridge modalities so that text descriptions can serve as classifiers for visual inputs.
- diagnose when transfer fails because source and target distributions are too far apart.

THE FUNDAMENTAL QUESTION this chapter answers is how a model trained on one corpus can serve a task it never saw, with little or no additional supervision, and how to pick the right adaptation strategy for the data and compute budget at hand.

¹ A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021. <https://arxiv.org/abs/2103.00020>

² K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023. <https://arxiv.org/abs/2302.12173>

SELF-REFLECTION questions to guide your thinking:

- Why does one shot or few shot learning on top of a frozen backbone work, when training the same network from scratch on the same handful of examples does not?
- What role do class centroids play as anchor points in the embedding, and why does averaging many embeddings into one prototype still represent the class faithfully?
- In Lampert zero shot, the bridge from attributes to centroids is fitted only on the seen classes, yet it locates unseen classes correctly. What does the bridge actually learn that lets it generalise, and what role does source breadth play in making that work?
- Why does the Lampert bridge stall when two source classes share an identical attribute row, and what does that aliasing tell you about where the ceiling on attribute based zero shot lives?
- What changes when the hand built bridge is replaced by a contrastively trained text encoder, and why is the resulting structure of two encoders meeting in a shared space the same shape as Lampert?
- Why does cosine similarity work cleanly as the decision rule in both Lampert and CLIP, and what does it imply about how the two sides of the comparison have to be normalised?
- Why does composing pretrained pieces at inference time, a frozen backbone with a learned bridge and a retrieval or clustering step, produce useful behaviour on tasks that none of the pieces was specifically trained for?
- The CLIP contrastive loss encourages the diagonal of the image caption similarity matrix to dominate each row and each column. Why does that single objective produce both the alignment of matched pairs and the separation of unmatched ones at the same time, without a separate negative term?
- Why is the classification head discarded the moment you transfer to a new task, and what does the act of throwing it away tell you about where the value of pretraining actually accumulates inside the network?
- Why does the choice between freezing the backbone and fine tuning it end up depending on both how much labelled target data is available and how far the target distribution sits from the source, rather than being a fixed best practice?

RECAP of Key Concepts:

- A backbone is a pretrained encoder reused across tasks while the head supplies only the decision, so pretraining cost is amortised across many downstream uses rather than paid each time.
- Linear probes succeed at one and few shot because the frozen embedding has already separated the classes, so the probe only has to pick a side of a hyperplane that the backbone effectively pre-drew.
- Class centroids act as anchor points in the embedding: averaging the embeddings of a class gives a single prototype, and classification reduces to nearest centroid under cosine similarity once both sides are unit normalised.
- Zero shot means reaching unseen classes through a bridge between class semantics and image features, hand built and fitted by least squares in Lampert, learned contrastively from natural language pairs in CLIP; the bridge structure of two encoders meeting in a shared space is the same in both, only the fitting criterion changes.
- Foundation models are backbones whose scale and breadth push the embedding to cover downstream input distributions, so composition of frozen pieces at inference time can replace task specific training, and transfer only fails when source and target distributions diverge beyond what the embedding describes.

THE BACKBONE ARRIVES READY-MADE. Throughout this chapter we have taken pretrained encoders as given and asked how to reuse them. ImageNet supervised pretraining works for natural images because someone else once labelled fourteen million examples, but no second ImageNet is being built for every new domain. Foundation models need to find a source of supervision that hides in plain sight.

SELF-SUPERVISED LEARNING generalises this idea. Instead of asking humans for labels, the model is given a task that can be solved using only the structure of the data itself: predict an image from itself, predict the next token from the previous ones. The labels come from the data, and the representations that emerge are the very backbones this chapter has taught us to reuse.

TEASER. Where in the data does implicit labels, cues and supervision hide?

FEEDBACK

Self-Supervised Learning

THERE IS FREE LUNCH.

The Why

IN THE PREVIOUS CHAPTER we learned to reuse a pretrained backbone across tasks, and that foundation models extend reuse to tasks they were never explicitly trained on. This chapter generalises that idea, learning representations from raw data through a family of objectives that scale to the corpora behind today's foundation models.

LABELS $\tilde{y}$ ARE THE BOTTLENECK. The internet, sensor networks, and scientific instruments produce orders of magnitude more raw data than any annotation budget can keep up with. Annotation is expensive, biased by the curator, slow to scale with model size, and tied to the specific domain that worked them out.

SELF-SUPERVISED LEARNING EXTRACTS SUPERVISION FROM THE DATA ITSELF. Instead of asking a human what an image contains, we design a pretext task whose label can be read off directly from the data structure.

- **PRETEXT TASKS.** The pretext task is scaffolding; the representations the network builds while solving it are the real product, and they transfer to downstream tasks the network was never trained for. The first generation of self-supervised methods built auxiliary objectives by hand: Rotate the image and predict the rotation, scramble the patches and reassemble them, fill in the colour of a grayscale photo.
- **CONTRASTIVE LEARNING.** A second generation drops the hand-designed task and asks the network to recognise the same image under different views. Two random crops of one photo should

CORPUS. A dataset is curated and usually labelled, assembled with a specific task in mind. A corpus is the body of raw data itself, often scraped or harvested at web scale, with no task attached. Self-supervised learning is what lets a corpus take the place of a dataset.

IMAGENET absorbed years of human effort to assemble fourteen million labelled images. The open web holds a hundred billion images that no curator will ever touch. The asymmetry between labelled and unlabelled data is the structural fact that motivates this chapter.

J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009. DOI: 10.1109/CVPR.2009.5206848. <https://scholar.google.com/scholar?q=Deng+Dong+Socher+Li+Li+Fei-Fei+2009+ImageNet+large-scale+hierarchical+image+database>

Chapter 6 showed one route to such a backbone without labels: The variational autoencoder distilled a representation from a reconstruction objective alone.

land near each other in the embedding space, while crops from different photos should land apart.

MASKED AND NEXT-TOKEN PREDICTION. The dominant pretext task at scale is to mask part of the input and ask the model to reconstruct or denoise it. This can be masked tokens in BERT ³, masked patches in MAE, or the next token in autoregressive language models.

SPAN MASKING. Single-token masking is often trivial, since twenty visible neighbours give away the missing token. Span masking removes contiguous chunks instead, forcing the model to reconstruct multi-token structure. SpanBERT and T5 made this the default in modern encoder-decoder pretraining.

CAUSAL NEXT-TOKEN PREDICTION. The decoder-only language model takes the same denoising principle and applies it strictly left to right. At every position, the model sees only the prefix and predicts the next token. GPT scales this objective to internet-sized corpora, and the resulting model is a foundation model whose representations transfer to almost every language task by prompting alone.

PREDICTING IN LATENT SPACE. Reconstructing pixels, or tokens, wastes capacity on texture and lighting the downstream task does not care about. Joint-embedding predictive architectures (JEPA) move the target into feature space instead. The teacher and the student share the same encoder, the teacher's weights are an exponential moving average of the student's to stabilize targets over time, and no labels need to enter the pipeline at any point. On every step the teacher runs a forward pass on the unmasked input and produces target representations of the masked region. The student sees a masked view of the same input sample, predicts those target representations in the latent space, and the loss is the distance between predicted and target features. There is no decoder and pure prediction happens in the encoder's feature space, so the method is efficient and scales well to large models and large corpora.

IN ORDER TO MOVE FROM EXPENSIVE MANUAL ANNOTATION TO LEARNING FROM RAW DATA we have good reason to find ways to,

- design auxiliary tasks whose supervision signal is inherent in the data itself, so that scale is bounded by storage rather than by human effort.
- learn representations that capture semantic structure without any human labels, transferable across downstream tasks the model was

³ J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 4171–4186, 2019. <https://arxiv.org/abs/1810.04805>

EXAMPLE. Single-token: She slipped on a yellow [?] peel and fell over. Span: She slipped on a [? ? ?] and fell over, where the contiguous chunk is the three adjacent tokens yellow banana peel the model has to recover together.

IN THE BEGINNING WAS THE WORD. Prompt injection, reasoning, and agentic tool use.

CHOOSE BY REGIME. Architecture tracks the downstream task: Encoder-only for understanding, decoder-only for generation, encoder-decoder such as T5 and BART for both at doubled cost.

never explicitly trained for.

- recognise denoising as the unifying objective behind masked-token, masked-patch, and next-token prediction.
- match the masking pattern to the downstream regime, choosing between encoder-only, decoder-only, and encoder-decoder recipes when the task is understanding, generation, or both.
- predict in feature space rather than pixel space when raw reconstruction would spend capacity on signal the downstream task does not care about.

THE FUNDAMENTAL QUESTION this chapter answers is whether the structure of unlabelled data carries enough signal to teach a model what to look at, without any human ever telling it what to value.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thinking:

- What goes wrong when a pretext task is too easy, and how can the network exploit shortcuts that solve the task without producing useful representations?
- Why does the contrastive recipe of treating different views of the same input as similar, and views of different inputs as dissimilar, give a stronger signal than handcrafted pretext tasks such as rotation prediction?
- Why is the choice of what counts as a different view the design decision that determines which features the contrastive representation will treat as invariant?
- Why is denoising a useful frame for thinking about masked-token, masked-patch, and next-token prediction together rather than as separate methods?
- Where does the supervision signal come from in masked prediction?
- How does the architectural choice between encoder-only, decoder-only, and encoder-decoder follow from whether the downstream task is understanding, generation, or both?
- Why does the encoder-decoder recipe end up paying for both halves of the bargain in architecture cost?
- Why does masking a contiguous span force the model to learn more than masking the same number of scattered single tokens?
- Why is moving the prediction target from pixel space to feature space a useful trade-off when raw reconstruction would spend capacity on signal the downstream task does not care about?
- What role does the slow-moving averaged teacher play in producing a stable target the student can predict against, and why is the same loss not workable when the teacher equals the student at every step?
- Why is the latent-space prediction recipe still considered self-supervised even though there is a teacher network feeding the student?
- Across all the methods in this chapter, what is the common idea that lets each of them produce useful representations from raw data with no human labels in the loop?

RECAP of key concepts:

- Self-supervised learning derives its supervision from the structure of the data itself, with the pretext task as scaffolding and the learned representation as the product.
- Pretext tasks evolved from handcrafted recipes such as rotation prediction and patch scrambling toward principled relational objectives such as contrastive learning over augmented views.
- Denoising unifies the dominant masked-prediction family: Hide part of the input as masked tokens, masked patches, contiguous spans, or the next token in a sequence, and train the model to reconstruct what was hidden.
- The architectural choice between encoder-only, decoder-only, and encoder-decoder tracks the downstream regime, with encoder-only for understanding, decoder-only for generation, and encoder-decoder for both at the cost of doubled architecture.
- Predicting in latent space sidesteps the irrelevant signal that raw pixel reconstruction would spend capacity on, with an exponential moving average teacher providing a stable target the student can predict against in feature space.

SELF-SUPERVISED LEARNING SOFTENED THE ANNOTATION BOTTLE-NECK FROM ONE SIDE. Instead of asking humans to label more data, the chapter designed objectives and tasks that read supervision off the data structure itself, and the resulting representations thus can be transferred to downstream tasks the model was never explicitly trained for. The bottleneck has another side, where supervision for dedicated downstream tasks still arrives, from humans, from auxiliary sensors coupled to the input, or from larger models standing in as labellers, but in the form of rough rules, heuristics, proxy signals, or noisy stand-in labels rather than clean per-example annotations.

THE NEXT CHAPTER TURNS TO WEAK SUPERVISION, where a label model learns to combine the noisy and conflicting rule outputs into a single training signal. Self-supervised learning trusted the structure of the data; weak supervision trusts the structure of the rules; both share the conviction that some signal is a signal.

TEASER. A self-supervised backbone gives you a head start, not a finished classifier; the downstream task still wants labels. What signals and cues can you think of to put in the place of supervision.

FEEDBACK

Weak Supervision

A WEAK SIGNAL, IS STILL A SIGNAL.

The Why

IN THE PREVIOUS CHAPTERS we have seen how to represent generalizable structure off the data, and defaulted to small clean head bolted on the embedding. This chapter turns to the head itself, asking what to do in the absence of clean labels.

WEAK LABELS ARE DRAWN FROM HETEROGENEOUS SOURCES, each with its own coverage and reliability profile. Crowd annotations exhibit substantial inter-annotator disagreement. Heuristic rules attain high precision on cases anticipated by their author and provide low coverage elsewhere. Large Pretrained classifier models afford broad coverage but are systematically overconfident on inputs lying outside the training distribution. Physical sensor measurements and behavioural proxies serve as auxiliary signals, correlating with downstream events of interest without themselves constituting labels, yet both can be folded into the pipeline as labelling-function outputs. The composition of these sources varies by modality, with text, image, sequence and tabular settings each exhibiting characteristic patterns. A unifying abstraction subsumes all of them: A labelling function is a programmatic mapping from input to a label. Under this representation, disparate sources reduce to a single object class amenable to downstream or bidirectional aggregation.

A JUDGE IS A LABELLING FUNCTION THAT EXPLOITS THE VERIFIER-GENERATOR ASYMMETRY. Discriminating a good answer from a poor one is structurally easier than producing a good one from scratch, the same gap that powers GAN discriminators and RLHF reward models. A language model judge consumes a candidate output and returns a scalar or categorical verdict ⁴, sitting alongside symbolic and statistical labelling functions under the same uniform interface.

LABELS ARE RANDOM VARIABLES

$\tilde{y}$. Not ground truth facts, just noisy realisations of an underlying truth y that we never observe directly. The annotation pipeline is a measurement device, and like every measurement device it has a bias and a variance.

HEURISTICS AS BENCHMARKS. Cheap to author, interpretable in their failures, and immune to training-distribution shift, heuristic baselines establish the performance floor a learned method must clear. The gap between a heuristic and a learned model quantifies whether the additional complexity is justified.

SENSORS AND PROXIES. Sensors such as vibration, touch, torque, current draw, lidar and you name it serve as proxies for machine failure, anomaly onset, or obstacle presence. Behavioural signals such as clicks, dwell times, completion rates, and purchase records serve as proxies for relevance, engagement, or user preference.

DIE ODE. Try writing a four-line ode to a banana in the manner of Schiller. Then judge whether this one is any good:

Sei mir begrüßt, du krumme Frucht,
in Gold und Schweigen eingehüllt;
es trägt die Reife stille Wucht,
bis süß dein Schicksal sich erfüllt.

⁴ L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks*, 2023. <https://arxiv.org/abs/2306.05685>

Judges offer lower cost than human evaluation and broader coverage than regular-expression rules, while inheriting the calibration and stylistic biases of the underlying model. Process reward models generalise the construction to intermediate reasoning steps, scoring each step of a chain rather than only the final answer and so spreading supervision along the whole trajectory instead of concentrating it at the end.

GENERATORS ARE LARGE MODELS THAT PRODUCE LABELLED EXAMPLES DIRECTLY. Where a judge discriminates, a generator synthesises both input and label. The canonical case is knowledge distillation: A teacher’s softmax supplies richer supervision than a one-hot target, and a smaller student is trained to match it. The same recipe underlies synthetic-data and instruction-tuning pipelines at scale.

SOFT LABELS ARE THE NATURAL OUTPUT OF WEAK SUPERVISION. A soft label assigns a probability to each candidate class, preserving inter-class structure that an argmax would discard. In Snorkel-style pipelines this distribution is the posterior of the label model, which sees only the labelling-function outputs and never the inputs themselves. The end model, a high-capacity discriminator, trains on those soft posteriors with full access to the inputs but never sees the labelling functions, and so generalises to inputs on which no labelling function ever fired. The same soft-label construction recurs across judges, distillation, and label smoothing, where the source of the distribution differs but the training objective is identical.

IN ORDER TO MOVE FROM THE FICTION OF GROUND TRUTH LABELS TO PRINCIPLED LEARNING UNDER NOISE we have good reason to find ways to,

- collect rules, crowd annotations, pretrained classifiers, and sensor proxies behind one labelling-function interface.
- exploit the verifier-generator gap with language-model judges as discriminators over candidate outputs.
- use larger models as generators, distilling a teacher’s softmax into a smaller student.
- aggregate noisy votes through a label model and train an end model on the resulting soft posteriors.

THE FUNDAMENTAL QUESTION this chapter answers is how to compose imperfect sources into supervision a model can trust.

AUXILIARY TASKS construct intermediate and spreaded supervision. If the main task is hard to label, find an easier auxiliary task that shares structure with it and train on that instead.

SNORKEL. Three moving parts: Engineers write labelling functions in Python that each vote on a data point or abstain, a label model learns the accuracies and correlations of those functions to turn their votes into a probability distribution over the true label, and a discriminative end model is trained on those probabilistic labels and generalises beyond the rules.

A. Ratner, S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Ré. Snorkel: Rapid training data creation with weak supervision. In *Proceedings of the VLDB Endowment*, volume 11, pages 269–282, 2017. <https://arxiv.org/abs/1711.10160>

ACTIVE SUPERVISION. The supervised complement to weak supervision: A small budget of clean labels is spent on the inputs from which the model stands to gain the most. Strictly supervised learning, but with the selection strategy as the lever. Acquisition criteria such as uncertainty sampling, query-by-committee, and expected model change rank candidates by anticipated information gain, and an annotator is queried only for the highest-ranked.

B. Settles. *Active Learning*. Morgan & Claypool Publishers, 2012. <https://scholar.google.com/scholar?q=Settles+2012+active+learning+synthesis+lectures>

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thinking:

- Why are labels best treated as random variables $\tilde{y}$ rather than as ground truth, and what does that reframing buy you when you design a training pipeline?
- Why is it useful to treat heterogeneous label sources, from heuristic rules and crowd annotations to pretrained classifiers and sensor proxies, behind a single labelling-function interface, and what does the abstraction hide?
- When does a sensor measurement or a behavioural signal qualify as a labelling function, and when does the correlation it relies on fall apart?
- What is the verifier-generator gap, and how do GAN discriminators, RLHF reward models, and LLM-as-judge all sit on the same asymmetry?
- What calibration and stylistic biases does an LLM-as-judge inherit from its base model, and what does that mean for using it as a labelling function at scale?
- How does a process reward model spread supervision along a reasoning chain rather than concentrating it on the final answer, and why does that matter for training?
- Why does knowledge distillation transfer more information through a teacher's softmax than through one-hot labels, and where does the same recipe show up at scale?
- How does the label model in Snorkel recover each labelling function's accuracy and correlation structure without ever observing the true label?
- Why does the end model generalise to inputs on which no labelling function ever fired, and what would break this property?
- What do label smoothing, distillation, and label-model posteriors share, and where do they differ?
- Across rules, judges, distilled teachers, and label models, what is the common move that turns noisy partial signal into supervision a model can trust?

RECAP of key concepts:

- Labels are random variables, not ground truth; any noisy source, heuristic rule, crowd worker, pretrained classifier, sensor measurement, or behavioural proxy, is unified behind one labelling-function interface.
- Judges exploit the verifier-generator gap: discriminating a good output from a poor one is structurally easier than producing a good one, and LLM judges, GAN discriminators, and RLHF reward models all sit on the same asymmetry. Process reward models extend the construction to intermediate steps, scoring each step of a reasoning chain so that supervision is spread along the whole trajectory rather than concentrated on the final answer.
- Generators turn larger models into labellers; knowledge distillation transfers a teacher's softmax to a smaller student, and the same recipe powers synthetic-data and instruction-tuning pipelines.
- The label model aggregates labelling-function votes into a probabilistic posterior over the true label, sees only the votes and never the inputs, and a high-capacity end model trains on those soft posteriors with full input access and so generalises beyond the rules.
- Soft labels recur across label-model posteriors, distillation, and label smoothing; the source of the distribution differs, the training objective is identical.

WEAK SUPERVISION INTERFACED THE ANNOTATION BOTTLENECK FROM NOISY PARTIAL SUPERVISION. Instead of deriving supervision from the data structure as self-supervised learning did, the chapter accepted noisy labels from rules, crowds, sensors, judges, and larger models, and combined it through a label model into soft posteriors that an end model could train against. Throughout, the auxiliary model was a labelling model: It answered what class an input belonged to and nothing else.

THE NEXT CHAPTER TURNS TO INTERACTIVE AND REINFORCEMENT LEARNING, where the model's predictions turn into actions which interact with an environment, which proactively influences the supervision cues it receives. The supervision is no longer a static label per input, but a dynamic signal that depends on the model's behaviour and the environment's response.

TEASER. How does the supervision problem change when model interacts with an environment and receives supervision not as per-example labels but as action-level signals such as demonstrations, feedback, corrections, or rewards rather than per-example labels?

FEEDBACK

Part IV

Interaction Pattern

Reinforcement Learning Fundamentals

PLAYING ATARI.

The Why

IN THE PREVIOUS CHAPTER supervision still arrived as labels, but a label on a single input cannot transmit a sequence of decisions or the consequences of one action on the next. Reinforcement learning answers a complementary question: What if the supervision is a scalar reward and depends on the sequence of actions a model takes?

MARKOV DECISION PROCESSES. The setting is formalised ⁵ as a tuple of a state s , an action a , a transition probability $P(s' \mid s, a)$, a reward $r(s, a)$, and a discount factor $\gamma \in [0, 1)$. The Markov property requires that the next state depends only on the current state and action, which lets the agent reason forward without dragging the past along, while γ downweights rewards t steps in the future by γ^t , keeping the sum of returns finite over infinite horizons.

BELLMAN EQUATIONS. The value of a state under policy π is the expected discounted return when starting in s and following π from then on. Bellman's insight is that this value satisfies a local recursion: The value of a state equals the expected immediate reward plus the discounted expected value of the next state. This turns long-horizon optimisation into a fixed-point problem which suits Markovian dynamics.

VALUE-BASED METHODS LEARN WHAT EACH ACTION IS WORTH. A Q-learning agent maintains an action-value table and bootstraps each estimate against the next, converging to the optimum even when behaviour is off-policy. The Deep Q-Network, DQN, scales the table to a neural network for high-dimensional inputs, which means a single set of weights now produces the value for every state-action pair, and updating the weights for one state changes the values for all

REWARD. A scalar number returned by the environment after an action, carrying no information about which action would have been better, no gradient, no class identity, only a magnitude and a sign. Let's extract signal yet again.

SIMULATION AS SUBSTRATE. Most reinforcement learning happens inside a simulator, since collecting millions of transitions on real hardware is prohibitive. The simulator's reward and transition function are perfect inside the sim and biased outside it, and the resulting sim-to-real gap is the central practical concern.

⁵ R. Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, 1961. <https://scholar.google.com/scholar?q=Bellman+1961+adaptive+control+processes+guided+tour>

PRESSURE ONE: CREDIT ASSIGNMENT. The reward arrives long after the action that helped earned it. Bellman decomposition turns the long-horizon problem into a local fixed-point one.

MODEL AND POLICY. A model θ is the bag of parameters that gradient descent updates. A policy π is the behaviour rule the agent acts under, mapping states to actions. The two coincide in policy-gradient methods, where θ directly parameterises π_θ , but separate in value-based methods, where θ parameterises a value function Q_θ and the policy is derived from it as greedy or ϵ -greedy.

of them. Two instabilities follow.

REPLAY BUFFER First, the agent's experience is a stream of correlated transitions: Each new state is the consequence of the previous one, and the agent often spends long stretches in similar regions of the environment. A network trained directly on this stream overfits to whatever region the agent is currently in and forgets what it learned about earlier ones. The experience replay buffer stores past transitions in a large pool and samples minibatches uniformly from it, so each gradient step sees a mix of transitions from many different points in time. Notice that this quickly becomes a compute bottleneck and does not scale.

TARGET NETWORK Second, the bootstrapped target uses the same network whose weights we are updating, so each update changes both the predictions and the targets they are regressing against, and convergence is unstable because the network is chasing a target that moves with it. A target network is a slow-updating copy used only to compute that target, kept frozen for thousands of steps so the online network has a stable regression problem to converge to before the target shifts.

POLICY-BASED METHODS LEARN THE ACTION RULE DIRECTLY. A value-based method learns what each action is worth and then acts greedily, deriving the policy from the value estimates. A policy-based method skips the value function and parameterises a stochastic policy $\pi_\theta(a \mid s)$ that outputs the probability of each action directly. This also allows for continuous action spaces, where a value-based method regresses. The policy gradient theorem turns the long-run expected return into a quantity we can backpropagate against, and applies it by rolling out a trajectory, observing the return, and nudging π_θ to make the actions that led to high return more likely and those that led to low return less likely.

TRUST REGIONS AND CLIPPING. Policy-gradient methods are on-policy: The gradient is computed under the old policy, so if a single update moves the new policy too far the data no longer applies and performance collapses. Trust-region methods address this by explicitly constraining the size of the policy update, which requires a second-order optimisation step that is expensive to compute. Proximal Policy Optimisation, PPO⁶, replaces the explicit constraint with a much simpler clipping trick. Each action in a trajectory has a probability under the old policy and a probability under the new policy, and their ratio measures how much the policy has shifted on that

PRESSURE TWO: STABILITY. Bootstrapped value estimates and policy gradients can amplify their own noise. Replay buffers, target networks, and trust regions stop the loop from running away.

OFF-POLICY VS ON-POLICY. Q-learning and DQN learn from data collected under any behaviour, including a replay buffer of past transitions. Policy-gradient methods need data collected under the current policy, which is why trust regions matter: They control how far the new policy can drift before the old data stops applying.

⁶ J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. <https://arxiv.org/abs/1707.06347>

action since the last update. PPO clips this ratio to a narrow band around one, typically $[0.8, 1.2]$, before plugging it into the policy-gradient update. The width of the band opts for stability over sample efficiency.

TRADE-OFF EXPLORATION AND EXPLOITATION. Every method above needs a behaviour rule for collecting data, and that rule has to balance two pressures. Exploit what is currently known to be good so reward is collected, explore actions that are uncertain so the value estimates can improve. ϵ -greedy, and entropy bonuses on the policy are the instruments that show up everywhere in practice, and the design of better exploration is an open subfield in its own right.

IN ORDER TO MOVE TO LEARNING FROM A SCALAR, MODEL DEPENDANT REWARD we have good reason to find ways to,

- formalise sequential decision problems as Markov Decision Processes and decompose long-horizon value into a local Bellman recursion.
- learn action values by bootstrapping each estimate against the next, and stabilise the neural-network case with a replay buffer for decorrelated samples and a target network for a fixed regression target.
- parameterise a stochastic policy directly and update it by rolling out trajectories and reweighting actions according to the returns they led to.
- constrain how far each policy update is allowed to move from the previous one, so that the data collected under the old policy still informs the new one.
- collect informative experience under uncertainty, balancing exploitation of known reward against exploration of unfamiliar actions.

THE FUNDAMENTAL QUESTION this chapter answers is whether a single scalar reward, delayed and sparse, carries enough signal to teach an agent how to act, without any teacher ever specifying what the right action would have been.

PRESSURE THREE: EXPLORATION.

Without enough variety in the data, the agent never sees the actions whose value it has yet to estimate. Exploration is the open subfield that supplies that variety.

ATARI AND GO. A DQN agent learned to play forty-nine Atari games from raw pixels and a single score signal. The score arrives at the end of a life or an episode, not after each frame, and the agent has to figure out by itself which of the thousands of intermediate joystick presses earned it.

V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. DOI: 10.1038/nature14236. <https://scholar.google.com/scholar?q=Mnih+2015+human+level+control+deep+reinforcement+learning>

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thinking:

- Why does the Markov property let the agent reason about the future without dragging the past along, and what would change if the next state depended on the full history?
- Why does the discount factor γ exist at all, and how does $\gamma < 1$ keep the sum of returns finite over infinite horizons?
- How does the Bellman recursion turn a long-horizon optimisation into a local fixed-point problem?
- What does it mean for Q-learning to be off-policy, and why does that property let DQN learn from a replay buffer of past transitions?
- Why does scaling the Q-table to a neural network introduce instabilities that the tabular case did not have, given that one set of weights now produces values for every state-action pair?
- What problem does the experience replay buffer solve, how does uniformly sampling past transitions from a large pool address it, and where does the approach run into a compute bottleneck?
- Why is a target network needed on top of the replay buffer, and what goes wrong when the target moves with the network being trained?
- How is a policy-based method different from a value-based method, why does parameterising $\pi_\theta(a \mid s)$ directly let you skip learning the value function, and how does this open the door to continuous action spaces?
- Why is the policy gradient theorem the key technical move that lets us treat the long-run expected return as a quantity we can backpropagate against?
- Why are trust regions and PPO clipping a policy-gradient concern only, and not relevant for Q-learning?
- Why does PPO clip the ratio between new and old policy probabilities to a narrow band such as $[0.8, 1.2]$, and how does the width of that band trade stability against sample efficiency?
- Why does every method in this chapter need a behaviour rule for exploration, and what role do ε -greedy, entropy bonuses on the policy, and stochastic policies play as the three blunt instruments?

RECAP of key concepts:

- Markov decision processes formalise sequential decision making as the tuple (s, a, P, r, γ) with the Markov property: the next state depends only on the current state and action, and γ downweights rewards t steps in the future by γ^t to keep returns finite over infinite horizons.
- The Bellman equations decompose the value of a state into immediate reward plus discounted value of the next state, and almost every method in this chapter is a way to solve that local fixed-point problem under different practical constraints.
- Value-based methods learn what each action is worth and act greedily; Q-learning is off-policy and converges in the tabular case, and DQN scales it to neural networks with an experience replay buffer that mixes correlated transitions and a target network that holds the regression target still, though replay sampling becomes a compute bottleneck at scale.
- Policy-based methods skip the value function and parameterise a stochastic policy $\pi_\theta(a \mid s)$ directly, which also unlocks continuous action spaces where value-based regression struggles; the policy gradient theorem provides a backpropagatable surrogate, and PPO bounds the on-policy update by clipping the probability ratio between new and old policy to a narrow band around one.
- Exploration supplies the data variety the methods above consume; ϵ -greedy, entropy bonuses on the policy, and stochastic policies are the three blunt instruments, and better exploration is an open subfield in its own right.

REINFORCEMENT LEARNING'S HARDEST PROBLEM IS LONG-HORIZON CREDIT ASSIGNMENT. Bellman decomposition reduces it to local updates, but across sparse, long horizons it arrives too rarely to learn from on any practical budget.

TEASER. Bellman decomposition makes long-horizon credit assignment tractable in principle. In practice, sparse rewards mean infeasible many steps before any informative signal arrives. What signal would shortcut all that?

THE NEXT CHAPTER TURNS TO INTERACTIVE AND IMITATION LEARNING, where we compress the long-horizon credit-assignment problem into a single demonstration, correction, or preference. Reinforcement learning derived its supervision from environmental reward; interactive learning derives it from few-shot demonstrations, corrections, or preferences from an annotator.

FEEDBACK

Index

- active learning, [27](#), [28](#)
- actor-critic, [34](#)
- advantage function, [34](#)
- adversarial inputs, [18](#)
- Atari, [35](#)
- autoregressive language model, [22](#)
- auxiliary signal, [27](#)

- backbone, [17](#)
- baseline, [27](#)
- Bellman equation, [33](#)
- BERT, [21](#)
- bottleneck design, [17](#)

- CLIP, [17](#), [18](#), [21](#)
- contrastive learning, [21](#)
- corpus, [21](#)
- credit assignment, [33](#)
- cross-modal alignment, [18](#)

- distribution shift, [18](#)
- DQN, [33](#)

- embedding, [17](#)
- encoder-decoder, [22](#)
- exploitation, [35](#)
- exploration, [35](#)

- few-shot learning, [18](#)
- fine-tuning, [18](#)
- foundation model, [17](#)

- generalisation, [17](#)
- Go, [35](#)
- GPT, [18](#)

- I-JEPA, [22](#)
- ImageNet, [20](#)

- JEPA, [22](#)

- knowledge distillation, [28](#)

- label model, [28](#)
- label noise, [27](#)
- labeling function, [27](#)
- labeling functions, [27](#)
- large language model, [17](#)
- license, [2](#)
- LLM-as-judge, [27](#)

- Markov Decision Process, [33](#)
- masked prediction, [22](#)
- model, [33](#)
- multimodal, [18](#)

- negative transfer, [18](#)
- next-token prediction, [22](#)

- off-policy, [34](#)
- on-policy, [34](#)

- PEFT, [18](#)
- policy, [33](#)
- policy gradient, [34](#)
- PPO, [33](#), [34](#)
- pretext task, [21](#)
- pretraining, [17](#), [20](#)
- process reward model, [27](#)
- prompt injection, [18](#)

- Q-learning, [33](#)

- recap, [24](#), [29](#)
- reflection, [24](#), [29](#)
- REINFORCE, [34](#)
- Reinforcement Learning, [33](#)

- reinforcement learning, [33](#)
- replay buffer, [34](#)
- representation learning, [17](#)
- reward, [33](#)

- SAM, [18](#)
- self-distillation, [22](#)
- Self-Supervised Learning, [21](#)
- self-supervised learning, [20](#), [21](#)
- sim-to-real, [33](#)
- SimCLR, [21](#)
- simulation, [33](#)
- Snorkel, [27](#), [28](#)
- soft labels, [28](#)
- span masking, [22](#)
- stability, [34](#)
- synthetic data, [28](#)

- target network, [34](#)
- task-specific head, [17](#)
- teacher-student, [28](#)
- Transfer Learning, [17](#)
- transfer learning, [17](#)
- trust region, [34](#)

- V-JEPA, [22](#)
- VAE, [17](#)
- verifier generator gap, [27](#)
- Vision Transformer, [17](#)

- weak labels, [27](#)
- weak supervision, [27](#)

- Zero-Shot, [17](#)
- zero-shot learning, [18](#)